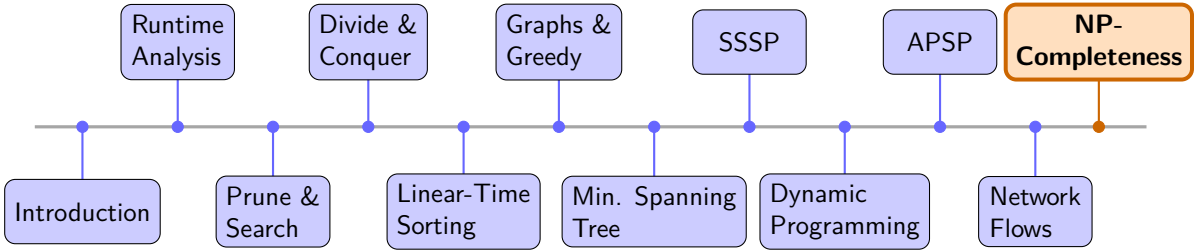
Algorithms

Dr. Mudassir Shabbir

LUMS University

Updated: May 9, 2026

Recap & What's Next



Approximation Algorithms

When Exact is Impossible, How Close Can We Get?

The Big Picture: Living with NP-Hardness

We have proved dozens of problems NP-complete. So what do we actually *do*?

Three main coping strategies

1. **Exact algorithms for small input**: branch-and-bound, ILP solvers, FPT.
2. **Heuristics**: no guarantee, but fast and often good in practice.
3. **Approximation algorithms**: polynomial time, *provable* quality guarantee.

Today: strategy 3. We want algorithms that run in $O(n^k)$ and return a solution whose objective value is *within a factor* of optimal.

Optimization Problems, Not Decision Problems

NP-completeness was defined for **decision problems** (yes/no answers).
Approximation algorithms operate on their **optimization counterparts**.

Decision problem	$\rightarrow$	**Optimization problem**
Does a VC of size $\leq k$ exist?	$\rightarrow$	Find the *minimum* vertex cover.
Does a tour of cost $\leq B$ exist?	$\rightarrow$	Find the *shortest* Hamiltonian tour.
Can m items of total value $\geq V$ fit?	$\rightarrow$	*Maximize* value in the knapsack.
Can all elements be covered by $\leq k$ sets?	$\rightarrow$	Find the *smallest* set cover.

We compare against OPT, the *true* optimum, even though we cannot compute it efficiently.

Optimization Problem: Formal View

An optimization problem is specified by:

- An instance I (input data).
- A set of valid (feasible) solutions $\mathcal{F}(I)$.
- An objective function $f_I(S)$ that assigns a numeric value to each feasible solution $S \in \mathcal{F}(I)$.

Objective Value of a Valid Solution

For any valid solution S, its **objective value** is exactly the number $f_I(S)$.

- Minimization: smaller $f_I(S)$ is better.
- Maximization: larger $f_I(S)$ is better.

Examples. Vertex Cover: $f_I(S) = |S|$ (minimize). Knapsack: $f_I(S) = \sum_{i \in S} v_i$ (maximize).

The Central Challenge: Bounding OPT

We want to prove $A(I) \leq \alpha \cdot \mathrm{OPT}(I)$. The catch:

We cannot compute OPT: it is NP-hard!

Strategy: find a *lower bound* on OPT (for minimization)

This is the core skill: find the *right* structural lower bound for the problem at hand.

Approximation Ratio?

Minimization problem

Algorithm A is a α-**approximation** ($\alpha \geq 1$) if for every instance I:

$$A(I) \leq \alpha \cdot \mathrm{OPT}(I).$$

Maximization problem

Algorithm A is a α-**approximation** ($\alpha \geq 1$) if for every instance I:

$$A(I) \geq \frac{\mathrm{OPT}(I)}{\alpha} \quad \text{(equivalently, } \mathrm{OPT}(I) \leq \alpha\, A(I)\text{)}.$$

Approximation Ratio Classes

- Constant ratio: e.g. 2, $3/2$, $8/7$.
- Ratio $1+\varepsilon$ for any $\varepsilon > 0$: **PTAS** (Polynomial-Time Approximation Scheme).
- Ratio $1+\varepsilon$ with runtime polynomial in both n and $1/\varepsilon$: **FPTAS**.
- $O(\log n)$ ratio: e.g. Set Cover.
- No *useful* approximation unless P $=$ NP: e.g. Clique.

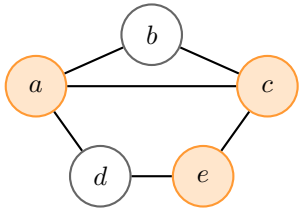
Vertex Cover: Reminder

Problem

Given an undirected graph $G = (V, E)$, find a **minimum vertex cover**: a smallest set $C \subseteq V$ such that every edge has at least one endpoint in C.

NP-hard. No poly-time exact algorithm (unless P = NP).

Example.



Lower bound (UGC): Cannot do better than 2.

Matching: Definitions

Matching

A **matching** in a graph $G = (V, E)$ is a set $M \subseteq E$ of edges such that no two edges in M share an endpoint.

Maximal Matching

A matching M is **maximal** if no edge can be added to M without violating the matching property (i.e., M is not a subset of any larger matching).

Maximum Matching

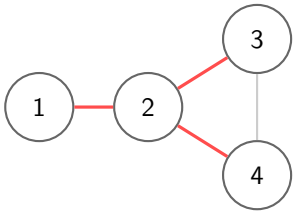
A matching M is **maximum** if it has the largest possible number of edges among all matchings. (Maximum $\Rightarrow$ maximal, but not vice versa.)

Perfect Matching

A matching M is **perfect** if every vertex in V is matched, i.e., $|M| = |V|/2$.

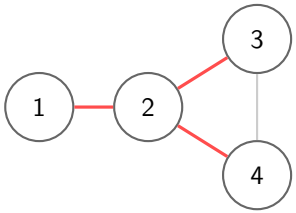
Is This a Matching?

Candidate A. Highlighted:
$\{(1,2),(2,3),(2,4)\}$.



Is This a Matching?

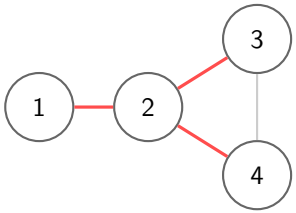
Candidate A. Highlighted:
$\{(1,2),(2,3),(2,4)\}$.



✗ Not a matching.
Vertex 2 is shared by all three edges.

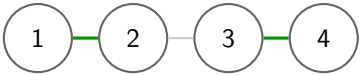
Is This a Matching?

Candidate A. Highlighted: $\{(1,2),(2,3),(2,4)\}$.



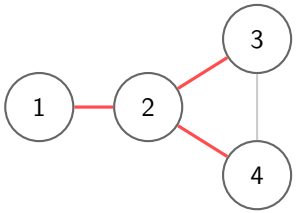
✗ Not a matching.
Vertex 2 is shared by all three edges.

Candidate B. Highlighted: $\{(1,2),(3,4)\}$.



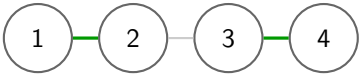
Is This a Matching?

Candidate A. Highlighted: $\{(1,2),(2,3),(2,4)\}$.



✗ Not a matching.
Vertex 2 is shared by all three edges.

Candidate B. Highlighted: $\{(1,2),(3,4)\}$.



✓ A matching.
No two highlighted edges share a vertex.

Maximal vs. Maximum

Graph: path $1{-}2{-}3{-}4{-}5{-}6$.

Candidate A: $M_A = \{(2,3),(4,5)\}$.

Matching? Maximum? Maximal? Perfect?

Maximal vs. Maximum

Graph: path $1{-}2{-}3{-}4{-}5{-}6$.

Candidate A: $M_A = \{(2,3),(4,5)\}$.

Matching? Maximum? Maximal? Perfect?

✓ Matching ✓ Maximal

✗ Not maximum ($|M_A| = 2$) ✗ Not perfect

Maximal vs. Maximum

Graph: path $1{-}2{-}3{-}4{-}5{-}6$.

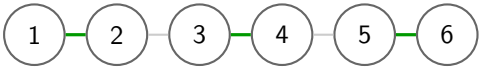
Candidate A: $M_A = \{(2,3),(4,5)\}$.

Matching? Maximum? Maximal? Perfect?

✓ Matching ✓ Maximal

✗ Not maximum ($|M_A| = 2$) ✗ Not perfect

Candidate B: $M_B = \{(1,2),(3,4),(5,6)\}$.



Matching? Maximum? Maximal? Perfect?

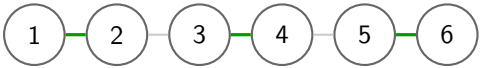
Maximal vs. Maximum

Graph: path $1{-}2{-}3{-}4{-}5{-}6$.

Candidate A: $M_A = \{(2,3),(4,5)\}$.

Matching? Maximum? Maximal? Perfect?
✓ Matching ✓ Maximal
✗ Not maximum ($|M_A| = 2$) ✗ Not perfect

Candidate B: $M_B = \{(1,2),(3,4),(5,6)\}$.

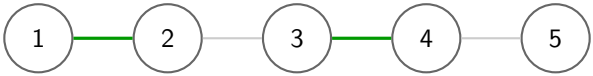


Matching? Maximum? Maximal? Perfect?
✓ Matching ✓ Maximal
✓ Maximum ($|M_B| = 3$) ✓ Perfect

Note: maximal $\not\Rightarrow$ maximum. But maximum $\Rightarrow$ maximal.

Maximum but Not Perfect

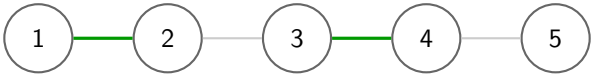
Graph: path $1{-}2{-}3{-}4{-}5$. Highlighted matching: $M = \{(1,2),(3,4)\}$.



Is M a matching? Is it maximum? Is it maximal? Is it perfect?

Maximum but Not Perfect

Graph: path $1{-}2{-}3{-}4{-}5$. Highlighted matching: $M = \{(1,2),(3,4)\}$.



Is M a matching? Is it maximum? Is it maximal? Is it perfect?

✓ **Matching** (no shared endpoints).

✓ **Maximum**: size 3 would need 6 endpoints, impossible with only 5 vertices.

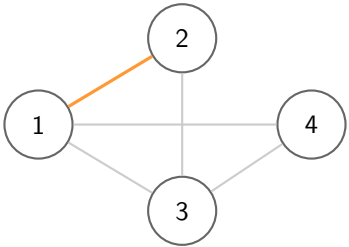
✓ **Maximal**: only free vertex is 5, and $(4,5)$ would conflict with 4.

✗ **Not perfect**: vertex 5 is unmatched.

Why? A perfect matching requires $|V|$ even. Here $|V| = 5$, so some vertex is always left out.

Is This Maximal?

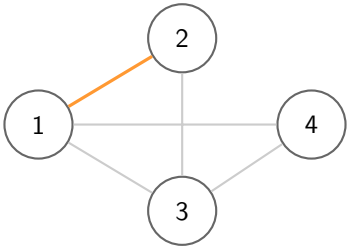
Graph: $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (1, 4), (2, 3), (3, 4)\}$. Highlighted: $M = \{(1, 2)\}$.



Is $M = \{(1, 2)\}$ a maximal matching?

Is This Maximal?

Graph: $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (1, 4), (2, 3), (3, 4)\}$. Highlighted: $M = \{(1, 2)\}$.



Is $M = \{(1, 2)\}$ a maximal matching?

✗ **Not maximal.** Edge $(3, 4)$ can still be added: vertices 3 and 4 are both free.

Matching Number and Vertex Cover: The Key Inequality

Fundamental Inequality

For any graph G, any matching M, and any vertex cover C:

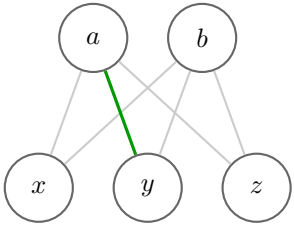
$$|M| \leq |C|.$$

Why? Each edge in M must be covered by C, and no single vertex can cover two matching edges (they share no endpoint). So C needs at least one distinct vertex per matching edge.

This gives us a nice bound on $\mathrm{OPT}_{\mathrm{VC}}$.

Matching Number and Vertex Cover: Example

Graph: $K_{2,3}$ (complete bipartite: top $\{a, b\}$, bottom $\{x, y, z\}$).



Max matching: $M = \{(a, y), (b, x)\}$, $|M| = 2$. (Add (b, x) to the picture.)

Our 2-approx VC: take both endpoints of each matching edge $\Rightarrow C = \{a, b, x, y\}$, $|C| = 4$.

Optimal VC: $C^* = \{a, b\}$: both top vertices cover all 6 edges! $|C^*| = 2$.

Ratio: $4/2 = 2$. This is the *worst case* for our algorithm on this graph.

König's theorem (bipartite only): $\nu(G) = \tau(G)$, i.e., max matching = min vertex cover. Here $\nu = \tau = 2$.

Vertex Cover:2-Approximation via Maximal Matching

Algorithm ApproxVC

1: $C \leftarrow \emptyset$, $E' \leftarrow E$
2: **while** $E' \neq \emptyset$ **do**
3: Pick any edge $(u, v) \in E'$
4: $C \leftarrow C \cup \{u, v\}$
5: Remove all edges incident to u or v from E'
6: **end while**
7: **return** C

Key observation. The selected edges form a *maximal matching* M. No two selected edges share an endpoint, so $|M|$ edges give $|C| = 2|M|$ vertices.

Theorem: ApproxVC is a 2-approximation.

$\mathrm{OPT} \geq |M|$ (matching lower bound), so $|C| = 2|M| \leq 2 \cdot \mathrm{OPT}$.

Vertex Cover 2-Approx: Worked Example

Graph: K_4 (complete on $\{1, 2, 3, 4\}$).

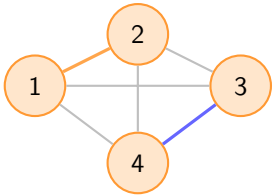
Step 1. Pick edge $(1, 2)$. Add $1, 2$ to C.
Remove all edges incident to 1 or 2.

Step 2. Remaining edges: $\{(3, 4)\}$. Pick $(3, 4)$.
Add $3, 4$ to C. Done.

$C = \{1, 2, 3, 4\}$, $|C| = 4$.

$\mathrm{OPT} = 3$ (e.g. $\{1, 2, 3\}$ covers all 6 edges).

Ratio achieved: $4/3 < 2$. ✓



Orange/blue = matched edges.
All 4 vertices added to C.

Worst case for this algorithm: a perfect matching on $2k$ vertices: gives $C = V$ but $\mathrm{OPT} = k$.

Load Balancing: Problem Definition

Problem: Makespan Minimization ($P\|C_{\max}$)

Given n jobs with processing times $p_1, p_2, \ldots, p_n > 0$ and m identical parallel machines.

Goal: Assign each job to a machine to **minimize the makespan** $C_{\max} = \max_i \sum_{j \text{ on machine } i} p_j$.

Two lower bounds on OPT:

$$\text{OPT} \geq \frac{1}{m} \sum_{j=1}^{n} p_j \qquad \text{(average load)}$$

$$\text{OPT} \geq \max_j p_j \qquad \text{(longest job must run somewhere)}$$

These two simple bounds are the foundation for most approximation analyses.

Load Balancing: List Scheduling (Greedy)

Algorithm ListSchedule

Process jobs $1, 2, \ldots, n$ in any order.
Assign each job to the machine with the **current smallest load**.

Example. $m = 3$ machines, jobs: $3, 4, 2, 5, 1, 2$.

Job	Time	Assigned to	Machine Loads
3	3	M1	[3, 0, 0]
4	4	M2	[3, 4, 0]
2	2	M3	[3, 4, 2]
5	5	M3	[3, 4, 7]
1	1	M1	[4, 4, 7]
2	2	M1 or M2	[6, 6, 7]

$C_{\max} = 7$. $\mathrm{OPT} = 6$ (assign $\{5, 1\}, \{4, 2\}, \{3, 2\}$). Ratio: $7/6 < 2$. ✓

List Scheduling: 2-Approximation Analysis

Theorem

ListSchedule is a 2-approximation for makespan minimization.

Proof.

Let j^* be the job that *finishes last* (determines $C_{\max}$).
Let T be the load on j^*'s machine *just before* j^* was assigned.

By greedy choice, every other machine had load $\geq T$ when j^* was assigned.
Summing over all m machines:

$$m \cdot T \;\leq\; \sum_{j \neq j^*} p_j \;\leq\; \sum_{j=1}^{n} p_j \;\leq\; m \cdot \mathrm{OPT}.$$

So $T \leq \mathrm{OPT}$. Therefore:

$$C_{\max} = T + p_{j^*} \;\leq\; \mathrm{OPT} + \mathrm{OPT} = 2\,\mathrm{OPT}.$$

Load Balancing: LPT Rule

Algorithm LPT (Longest Processing Time first)

Sort jobs so that $p_1 \geq p_2 \geq \cdots \geq p_n$.
Then run ListSchedule in this sorted order.

Theorem (Graham 1969)

LPT is a $\frac{3}{2}$-approximation for makespan minimization.

Intuition. Placing large jobs first leaves smaller jobs to fill gaps: less room for bad imbalance.

Key lemma (simpler). Under LPT, the last-finishing job j^* satisfies $p_{j^*} \leq \mathrm{OPT}/2$ (when $n > m$). Plugging into the List Scheduling analysis gives $C_{\max} \leq \mathrm{OPT} + \mathrm{OPT}/2 = 3\,\mathrm{OPT}/2$.

LPT: Why the $3/2$ Bound Holds

Claim: Under LPT with $n > m$, the last job j^* satisfies $p_{j^*} \leq \mathrm{OPT}/2$.

Proof sketch.

If $n \leq m$, every job can go to its own machine, so LPT is optimal. So assume $n > m$.

Under LPT, the last-finishing job is among the smaller jobs. At least $m + 1$ jobs have size $\geq p_{j^*}$. By pigeonhole, in any schedule some machine gets at least two of these jobs, so

$$\mathrm{OPT} \geq 2p_{j^*}.$$

Hence $p_{j^*} \leq \mathrm{OPT}/2$. Combining with the List Scheduling decomposition $C_{\max} = T + p_{j^*}$ and $T \leq \mathrm{OPT}$ gives

$$C_{\max} \leq \mathrm{OPT} + \frac{\mathrm{OPT}}{2} = \frac{3}{2}\,\mathrm{OPT}.$$

✓

LPT: Worked Example Showing Improvement

$m = 3$ machines. Jobs (sorted): $7, 5, 5, 3, 3, 2$.

List Scheduling (arbitrary order $2, 3, 3, 5, 5, 7$):

Job	Time	Machine
2	2	M1
3	3	M2
3	3	M3
5	5	M1
5	5	M2
7	7	M3

Loads: $[7, 8, 10]$. $C_{\max} = 10$.

LPT (sorted $7, 5, 5, 3, 3, 2$):

Job	Time	Machine
7	7	M1
5	5	M2
5	5	M3
3	3	M2
3	3	M3
2	2	M1

Loads: $[9, 8, 8]$. $C_{\max} = 9$.

$\mathrm{OPT} = 9$ (cannot do better: total load $= 25$, $\lceil 25/3 \rceil = 9$, job of size 7).

Set Cover: Problem

Problem

Universe $U = \{1, \ldots, m\}$, collection $\mathcal{S} = \{S_1, \ldots, S_n\}$ of subsets of U.

Goal: Find the smallest subcollection $\mathcal{C} \subseteq \mathcal{S}$ such that $\bigcup_{S \in \mathcal{C}} S = U$.

NP-hard (generalizes Vertex Cover).

Example. $U = \{1, 2, 3, 4, 5, 6\}$, four available sets:

$$S_1 = \{1, 2, 3, 4\}, \quad S_2 = \{2, 3, 5\}, \quad S_3 = \{4, 5, 6\}, \quad S_4 = \{1, 6\}.$$

Can we cover U with 2 sets? Yes: $\mathcal{C} = \{S_1, S_3\}$ covers $\{1, 2, 3, 4\} \cup \{4, 5, 6\} = U$. ✓

Lower bound on OPT. Any single set covers $\leq m$ elements, so $\mathrm{OPT} \geq \lceil m / \max_i |S_i| \rceil$.

Set Cover: Greedy Algorithm

Algorithm GreedyCover

1: $\mathcal{C} \leftarrow \emptyset$, $R \leftarrow U$ (remaining uncovered elements)
2: **while** $R \neq \emptyset$ **do**
3: Pick $S_i \in \mathcal{S}$ maximizing $|S_i \cap R|$
4: $\mathcal{C} \leftarrow \mathcal{C} \cup \{S_i\}$, $R \leftarrow R \setminus S_i$
5: **end while**
6: **return** $\mathcal{C}$

Greedy trace. $U = \{1, \ldots, 6\}$, sets $S_1 = \{1, 2, 3, 4\}$, $S_2 = \{2, 3, 5\}$, $S_3 = \{4, 5, 6\}$, $S_4 = \{1, 6\}$:

Round	Set picked	Elements covered	$\lvert R \rvert$ remaining
1	$S_1 = \{1, 2, 3, 4\}$	4 new	2
2	$S_3 = \{4, 5, 6\}$	2 new	0

Set Cover: $\ln |U|$-Approximation Analysis

Theorem

GreedyCover is an H_m-approximation, where $H_m = 1 + \frac{1}{2} + \cdots + \frac{1}{m} \approx \ln m$.

Proof sketch.

Let $\mathrm{OPT} = k^*$. At any point, the k^* optimal sets together cover all remaining elements, so the *best* single set covers at least a $1/k^*$ fraction of R.

If $|R| = r$ before a round, greedy picks $\geq r/k^*$ elements. After t rounds:

$$|R| \;\leq\; m\left(1 - \frac{1}{k^*}\right)^t \;\leq\; m\, e^{-t/k^*}.$$

Setting $|R| < 1$ (i.e. $R = \emptyset$) requires $t \leq k^* \ln m = \mathrm{OPT} \cdot \ln m$. ✓

Set Cover: Tightness and Hardness

Tight example. $m = 2^k$ elements. Arrange sets so each set covers exactly half the remaining uncovered elements. Greedy needs $k = \log_2 m$ rounds; if OPT $= 1$ (one big set covers all), ratio $= \log m = H_m$.

Hardness (Feige 1998)

No polynomial-time algorithm achieves ratio $(1 - \varepsilon) \ln m$ for any $\varepsilon > 0$, unless

$$\mathrm{NP} \subseteq \mathrm{DTIME}(n^{O(\log \log n)}).$$

Conclusion. Greedy is essentially optimal for Set Cover: we cannot do meaningfully better in polynomial time.

Set Cover: Worked Example

Instance where greedy is suboptimal (but still $\leq \ln m \cdot \mathrm{OPT}$).
$U = \{1, 2, 3, 4, 5, 6\}$, $\mathrm{OPT} = 2$.

$$S_1 = \{1, 2, 3, 4\},\ S_2 = \{1, 2, 5, 6\},\ S_3 = \{3, 4, 5, 6\}.$$

OPT: $\{S_1, S_3\}$ or $\{S_1, S_2\}$ or $\{S_2, S_3\}$: each covers U in 2 sets.

Greedy (breaking ties arbitrarily):

1. Pick S_1 (covers 4). $R = \{5, 6\}$.
2. Pick S_2 or S_3 (covers 2). $R = \emptyset$.

Greedy also gets 2 sets: optimal here.

Bad Example: Setup

Goal: Build an instance where greedy uses $\Theta(\log n)$ sets while $\mathrm{OPT} = 2$.

Universe. Take $2n$ elements split into two rows:

$$R_1 = \{a_1, \ldots, a_n\}, \qquad R_2 = \{b_1, \ldots, b_n\}.$$

Optimal cover (2 sets).

$$S^\star = R_1, \qquad T^\star = R_2 \quad \Longrightarrow \quad \mathrm{OPT} = 2.$$

Idea. Add carefully designed "trap" sets that look slightly better to greedy at each round.

Bad Example: Trap-Set Construction

Construct $C_1, C_2, \ldots, C_{\log_2 n}$ with

$$|C_i| \;=\; \frac{n}{2^{\,i-1}} \quad \text{(elements drawn from both rows).}$$

Each C_i is engineered so that, *at the moment greedy considers it*, it covers slightly *more* uncovered elements than $S^\star$ or $T^\star$ would.

Consequence.

$$\text{Greedy} = \log_2 n \;\text{ sets}, \qquad \mathrm{OPT} = 2.$$

So the approximation ratio is

$$\frac{\text{Greedy}}{\mathrm{OPT}} = \Theta(\log n).$$

Concrete Example ($n = 4$): Sets

Universe of size 8:

$$U = R_1 \cup R_2 = \{a_1, a_2, a_3, a_4, b_1, b_2, b_3, b_4\}.$$

Optimal cover ($\mathrm{OPT} = 2$):

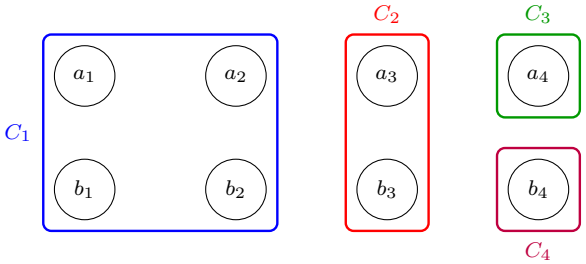
$$S^\star = \{a_1, a_2, a_3, a_4\}, \qquad T^\star = \{b_1, b_2, b_3, b_4\}.$$

Greedy's trap sets:

Step	Set	Size
1	$C_1 = \{a_1, a_2, b_1, b_2\}$	4
2	$C_2 = \{a_3, b_3\}$	2
3	$C_3 = \{a_4\}$	1
4	$C_4 = \{b_4\}$	1

Concrete Example ($n = 4$): Greedy Path

Greedy picks C_1, C_2, C_3, C_4 — 4 sets instead of 2.



Bad Example: Breaking Ties

The issue. Initially,

$$|C_1| = n = |S^\star| = |T^\star|,$$

so greedy could tie-break to $S^\star$ and finish in 2 steps.

Perturbation trick. Add one private element to each trap set:

$$C_i \longleftarrow C_i \cup \{x_i\},$$

where each x_i is fresh and appears nowhere else.
Then at round i:

$$|C_i \cap R| = \frac{n}{2^{i-1}} + 1 \quad > \quad \max\{|S^\star \cap R|, |T^\star \cap R|\} = \frac{n}{2^{i-1}}.$$

Hence greedy strictly prefers C_i every time.

Johnson (1974), Lovász (1975); Feige (1998): hardness near $(1 - o(1)) \ln n$.

Metric TSP: Problem

Metric TSP

Given n cities with pairwise distances $d_{ij} \geq 0$ satisfying the **triangle inequality**:

$$d_{ij} \leq d_{ik} + d_{kj} \quad \forall\, i, j, k.$$

Goal: Find a minimum-cost Hamiltonian cycle (tour).

Why the triangle inequality?
Without it, no constant-factor approximation exists (unless P = NP).

Reduction: given any graph G, set $d_{uv} = 1$ if $(u, v) \in E$, else $d_{uv} = \alpha n + 1$. A α-approximation would decide Hamiltonian Cycle: which is NP-hard.

So the triangle inequality is not just a convenience: it is *necessary* for any useful approximation guarantee.

Metric TSP: Known Results

Algorithm	**Ratio**	**Key tool**
MST + double tree	2	MST, Euler tour, shortcut
Christofides–Serdyukov (1976)	$3/2$	MST + min-weight matching
Karlin–Klein–Oveis Gharan (2020)	$3/2 - 10^{-36}$	Random spanning trees

Open problem. The true approximability threshold for metric TSP is unknown. No lower bound better than $123/122$ is known (assuming P $\neq$ NP).

Closing the gap between $3/2$ and the best lower bound is one of the most celebrated open problems in combinatorial optimization.

Metric TSP: 2-Approximation via MST

Algorithm MSTTOUR

1. Compute a minimum spanning tree T of the complete graph.
2. **Double** every edge of T to get an Eulerian multigraph T' (all degrees even).
3. Find an **Euler tour** of T' (visits every edge exactly once).
4. **Shortcut** repeated vertices: traverse the Euler tour, skipping already-visited cities.

Theorem: MSTTOUR is a 2-approximation.

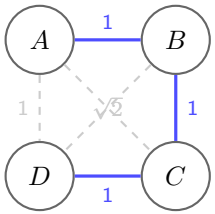
Analysis:

- Any tour $\Rightarrow$ spanning connected subgraph $\Rightarrow$ MST cost $\leq \mathrm{OPT}$.
- Doubling gives Euler tour of cost $2 \cdot w(T) \leq 2 \cdot \mathrm{OPT}$.
- Shortcuts never increase cost (triangle inequality): shortcut cost $\leq$ Euler tour cost.

Therefore: MSTTOUR cost $\leq 2 \cdot \mathrm{OPT}$. ✓

MST Tour: Worked Example

4 cities on a grid:



MST (blue): $A{-}B{-}C{-}D$, cost 3.

Double MST edges: each edge appears twice.

Euler tour: $A{-}B{-}C{-}D{-}C{-}B{-}A$ (cost 6).

Shortcut repeated B, C: Tour $A{-}B{-}C{-}D{-}A$, cost $1+1+1+1=4$.

$\mathrm{OPT}=4$ (same tour). ✓

Ratio $=1$ here (we got OPT).

In general: shortcuts exploit triangle inequality; ratio is at most 2.

Christofides Algorithm: 3/2-Approximation

Observation. The MST doubling step is wasteful. Can we get a cheaper Eulerian extension?

Algorithm CHRISTOFIDES (1976)

1. Compute minimum spanning tree T.
2. Let $O =$ set of odd-degree vertices in T. ($|O|$ is always even.)
3. Compute a **minimum-weight perfect matching** M on the vertices of O.
4. Combine $T \cup M$: all degrees now even: it is Eulerian.
5. Find an Euler tour; shortcut repeated vertices.

Theorem: Christofides gives a 3/2-approximation.

$w(T) \leq \mathrm{OPT}$. The odd vertices O can be split into two matchings using alternate edges of the OPT tour restricted to O; the cheaper matching costs $\leq \mathrm{OPT}/2$. So total cost $\leq \mathrm{OPT} + \mathrm{OPT}/2 = 3\,\mathrm{OPT}/2$.

Christofides: Matching on Odd Vertices

Why $w(M) \leq \mathrm{OPT}/2$.

Let $O = \{v_1, v_2, \ldots, v_{2k}\}$ be the odd-degree vertices in T, appearing in this cyclic order on the optimal tour.

The OPT tour restricted to O gives two perfect matchings:

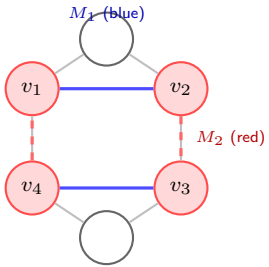
$$M_1 = \{(v_1, v_2), (v_3, v_4), \ldots\}, \quad M_2 = \{(v_2, v_3), (v_4, v_5), \ldots\}$$

By triangle inequality, $w(M_1) + w(M_2) \leq w(\mathrm{OPT})$.

So $\min(w(M_1), w(M_2)) \leq \mathrm{OPT}/2$.

The minimum perfect matching is at most this:

$$w(M) \leq \frac{\mathrm{OPT}}{2}.$$



Red vertices = odd-degree in T.

PTAS and FPTAS: Definitions

Polynomial-Time Approximation Scheme (PTAS)

A family of algorithms $\{A_\varepsilon\}_{\varepsilon>0}$ where for each fixed $\varepsilon > 0$, algorithm A_ε is a $(1+\varepsilon)$-approximation running in polynomial time in n.

Runtime *may be exponential* in $1/\varepsilon$: e.g. $O(n^{1/\varepsilon})$.

Fully Polynomial-Time Approximation Scheme (FPTAS)

Same as PTAS, but the runtime must be polynomial in **both** n **and** $1/\varepsilon$.

Example: $O(n^3/\varepsilon^2)$ is FPTAS. But $O(n^{1/\varepsilon})$ is not.

Hierarchy (strict inclusions):

$$\text{FPTAS} \subsetneq \text{PTAS} \subsetneq \text{constant approx} \subsetneq \text{poly approx.}$$

PTAS and FPTAS: Examples

Class	Problem	Note
FPTAS	0-1 Knapsack	Scaling DP
FPTAS	Subset Sum	Special case of Knapsack
FPTAS	Scheduling $P\|\|C_{\max}$	Fixed m machines
PTAS (not FPTAS)	Euclidean TSP	Arora 1998
PTAS (not FPTAS)	Max-k-SAT	Randomized
No PTAS	Set Cover $< \ln n$	Feige 1998
No PTAS	Clique	$n^{1-\varepsilon}$ hard
No constant	General TSP	Ham-Cycle reduction

Takeaway. FPTAS is the gold standard: you can get arbitrarily close to optimal in truly polynomial time. Most NP-hard problems do not admit an FPTAS.

Knapsack: FPTAS via Scaling

Problem

n items, weights w_i, values v_i; knapsack capacity W.
Goal: Maximize $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq W$.

Exact DP: $O(nV)$ where $V = \sum v_i$. Pseudo-polynomial: can be huge.

Algorithm KnapsackFPTAS(ε)

1. Let $v_{\max} = \max_i v_i$.
2. Scale values: $\hat{v}_i = \left\lfloor \dfrac{v_i \cdot n}{\varepsilon \cdot v_{\max}} \right\rfloor$.
3. Run exact DP on scaled values $\hat{v}_i$ (now all $\leq n/\varepsilon$).
4. Return the selected subset.

Runtime: $O(n \cdot \hat{V}) = O\left(\dfrac{n^3}{\varepsilon}\right)$. Polynomial in n and $1/\varepsilon$. ✓

Knapsack FPTAS: Correctness Analysis

Why ratio is $1/(1+\varepsilon)$?

Let $K = \dfrac{\varepsilon \cdot v_{\max}}{n}$. Then $\hat{v}_i = \lfloor v_i/K \rfloor$.

For any item i: $\quad v_i/K - 1 \le \hat{v}_i \le v_i/K$.

Let $S^* =$ optimal solution (exact values), $\hat{S} =$ FPTAS solution (scaled values).

Since $\hat{S}$ is optimal for scaled values: $\sum_{i\in\hat{S}} \hat{v}_i \ge \sum_{i\in S^*} \hat{v}_i$.

Translating back (at most n items, each scaled down by < 1):

$$\sum_{i\in\hat{S}} v_i \ge K \sum_{i\in\hat{S}} \hat{v}_i \ge K \sum_{i\in S^*} \hat{v}_i \ge \sum_{i\in S^*} v_i - nK = \mathrm{OPT} - \varepsilon \cdot v_{\max} \ge (1-\varepsilon)\,\mathrm{OPT}.$$

So FPTAS achieves at least $(1-\varepsilon)\mathrm{OPT}$ in $O(n^3/\varepsilon)$ time. ✓

Max-3SAT: Problem and Algorithm

Max-3SAT (Optimization)

Given a 3-CNF formula with m clauses (each clause has exactly 3 distinct literals).

Goal: Find an assignment that maximizes the number of satisfied clauses.

Algorithm: Random Assignment

Set each variable *independently* and *uniformly* at random:

$$\Pr[x_i = \text{True}] = \Pr[x_i = \text{False}] = 1/2.$$

Simplicity check. This is as simple as an algorithm can be: no problem structure used at all. Yet it achieves the best possible approximation ratio.

Max-3SAT: Analysis

How many clauses does random assignment satisfy?

A clause is *unsatisfied* only when all 3 of its literals evaluate to *false* simultaneously.

$$\Pr[\text{clause unsatisfied}] = \left(\frac{1}{2}\right)^3 = \frac{1}{8}.$$

$$\Pr[\text{clause satisfied}] = 1 - \frac{1}{8} = \frac{7}{8}.$$

By linearity of expectation, over all m clauses:

$$\mathbf{E}[\#\text{ satisfied clauses}] = \frac{7}{8}m \;\geq\; \frac{7}{8}\,\mathrm{OPT}.$$

Theorem

Random assignment is a $7/8$-approximation for Max-3SAT in expectation.

Max-3SAT: Derandomization

Can we make the algorithm deterministic?

Method of Conditional Expectations

Assign variables one by one. At each step, choose the value of x_i that keeps the *conditional expected* number of satisfied clauses $\geq 7m/8$.

Since the expectation starts at $\geq 7m/8$ and never drops below it, the final assignment satisfies $\geq 7m/8$ clauses deterministically.

This gives a *deterministic* polynomial-time $7/8$-approximation. ✓

Max-3SAT: Tightness

Is 7/8 the best possible?

Håstad's Inapproximability (1997)

For any $\varepsilon > 0$, it is NP-hard to approximate Max-3SAT to within a ratio of $7/8 + \varepsilon$.

Tight example. Take all $2^3 = 8$ possible 3-literal clauses over variables $\{x, y, z\}$:

$$(x \vee y \vee z),\ (x \vee y \vee \neg z),\ \ldots,\ (\neg x \vee \neg y \vee \neg z).$$

Every truth assignment satisfies exactly 7 of the 8 clauses: so $\mathrm{OPT}/m = 7/8$ exactly.

Conclusion. The trivial random algorithm is *essentially optimal*. No polynomial algorithm can do better.

Contrast. Max-2SAT: random gives $3/4$, tight by inapproximability; yet exact 2SAT is in P.

Inapproximability: Gap Reductions

So far: *upper bounds*: algorithms achieving ratio α.
Now: *lower bounds*: proving no poly-time algorithm can beat ratio α (unless P = NP).

Gap Reduction

To show no α-approximation exists for problem B, reduce from a hard problem A:

- **YES** instance of $A \Rightarrow B$ has a solution of value $\geq V$.
- **NO** instance of $A \Rightarrow$ every solution of B has value $< V/\alpha$.

A α-approximation for B would distinguish these two cases, solving A exactly: contradiction.

Example. General TSP inapproximability (proved earlier) is a gap reduction from Ham-Cycle with gap αn vs n.

Inapproximability: The PCP Theorem

Most inapproximability results for non-trivial ratios rely on a deep theorem:

PCP Theorem (Arora–Safra, Arora–Lund–Motwani–Sudan–Szegedy, 1992)

$$\mathrm{NP} = \mathrm{PCP}(O(\log n),\ O(1)).$$

Every NP problem has a *probabilistically checkable proof*: a verifier that reads only $O(\log n)$ random bits and $O(1)$ proof bits, yet accepts correct proofs with probability 1 and rejects incorrect proofs with probability $\geq 1/2$.

Why it implies inapproximability. The PCP theorem implies that Max-3SAT is hard to approximate above $7/8$, which then propagates via reductions to Vertex Cover, Set Cover, Clique, and many others.

It is one of the most influential results in theoretical computer science.

Inapproximability: Key Results

Problem	**Best ratio**	**Inapproximability**
Vertex Cover	$2 - O(1/\sqrt{\log n})$	$\geq 10\sqrt{5} - 21 \approx 1.36$ (UGC: 2)
Set Cover	$\ln n + O(1)$	$\leq (1 - \varepsilon) \ln n$ (unless P$\neq$NP)
Clique	$n^{1-\varepsilon}$	$n^{1-\varepsilon}$ (P$\neq$NP)
Max-3SAT	$7/8$	$7/8 + \varepsilon$
Metric TSP	$3/2$	No $\alpha < 123/122$ known
General TSP	:	No constant factor
Chromatic #	$n^{1-\varepsilon}$	No $n^{1-\varepsilon}$ approx

Unique Games Conjecture (Khot 2002): A widely-believed conjecture that (if true) would tighten many inapproximability results: e.g., showing Vertex Cover cannot be approximated below 2.

General TSP is Inapproximable

Theorem

For general (non-metric) TSP, no polynomial-time α-approximation exists for any constant α, unless P $=$ NP.

Proof. Reduction from Hamiltonian Cycle.
Given graph $G = (V, E)$, build TSP instance:

$$d_{uv} = \begin{cases} 1 & (u,v) \in E \\ \alpha n + 1 & (u,v) \notin E \end{cases}$$

- If G has a Ham. cycle: OPT tour has cost n.
- If G has no Ham. cycle: every tour uses ≥ 1 non-edge, costing $\geq n - 1 + (\alpha n + 1) > \alpha n$.

A α-approximation algorithm would distinguish cost n from cost $> \alpha n$: solving Ham. Cycle. ✓

Consequence: The triangle inequality is *necessary* for any meaningful TSP approximation.